

bond_pricing

September 20, 2026

1 Corporate bond pricing in OSEM

This notebook walks through how OSEM treats a corporate bond position: from raw input data, to projected cash flows, to a calibrated market-consistent price. Each corporate bond is defined by its characteristic, the market conditions such as spread and price.

On a very high level:

- 1) Each corporate bond is saved into a CorpBond object.
- 2) A portfolio of CorpBond objects is then saved into the CorpBondPortfolio.
- 3) Based on the CorpBondPortfolio and market conditions, the cash flows resulting from each bond are reconstructed into a matrix
- 4) Based on the cash flows and current market conditions, the z-spread is calculated using the bisection algorithm to return the current market price

1.1 Table of contents

1. Methodology
 - **Notation**
 - Step 1 Cash flow dates · Step 2 Date fractions · Step 3 Cash flow amounts
 - Step 4 Pricing off the risk-free curve · Step 5 Calibrating the z-spread
 - Step 6 Calibrating the whole portfolio · Step 7 Use in the full OSEM run
 - Generating new bonds
2. Preparation before running the notebook
 - Importing necessary external packages
 - Orient base folder
 - Import functions from the main code
 - Configuration and parameters
3. Corporate bonds
4. Projection of cash flows
 - Importing the information about the economic environment
 - **Sector spread input**
 - Cash flow projection of a bond portfolio
5. Pricing and calibration
 - Calculation of present value of each instrument
 - Step 4 in numbers: the gap the z-spread has to close
 - Calibrate the spread to match market price
6. Appendix: input data scheme

1.2 1. Methodology

How OSEM turns a bond description into a price, step by step. No code in this section; the sections that follow run each step on the input portfolio.

1.2.1 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
VD	valuation date, the date cash flows are discounted to ($VD = MD$ at $t = 0$)
E	end of the modelling window (<code>settings.end_date</code>)
I	issue date
M	maturity date
L	last cash flow date, $\min(E, M)$
N	notional amount
c	coupon rate, paid on each coupon date
f	coupon frequency (payments/year)
MV	market value (price) at the modelling date
z	bond-specific z-spread
$y(t)$	risk-free spot yield for maturity t , from the calibrated EIOPA curve

A note on f . The frequency sets only the *spacing* of the coupon dates, through `relativedelta(months=12 // f)`. It does **not** scale the coupon amount: c is the rate paid on each coupon date, not an annualised rate, so a bond with $c = 0.02$ and quarterly frequency pays 2% of the notional four times a year. A bond quoted with an annual coupon c_{ann} paid f times a year is entered in `Bond_Portfolio.csv` with `Coupon_Rate` = c_{ann}/f .

1.2.2 Step 1 — Cash flow dates

Every bond produces two kinds of cash flow: coupons and the return of notional.

A bond still alive when the modelling window closes is redeemed there at par, so both kinds of cash flow stop at

$$L = \min(E, M)$$

Notional repayment date:

$$t_M^d = L$$

Coupon dates:

$$I < t_1^d < t_2^d < \dots < t_k^d \leq L$$

The coupon grid is anchored on the issue date and stepped by $12/f$ months. Payments falling before MD are dropped, since only future cash flows are priced.

1.2.3 Step 2 — Date fractions relative to the valuation date

Only cash flows after the valuation date matter. Each remaining cash flow date is converted to a year fraction from it:

$$t_i = \frac{t_i^d - VD}{365.25}$$

At the modelling date $VD = MD$, which is the case this notebook demonstrates. In a full run the valuation date moves forward with every period and the fractions are re-measured from the new date.

This is what lets the discounting step treat every cash flow generically as “an amount, this many years from now.”

1.2.4 Step 3 — Cash flow amounts

Notional cash flow (paid once, at maturity):

$$cf_M = N$$

Coupon cash flows (paid at each coupon date):

$$cf_i = N \cdot c$$

1.2.5 Step 4 — Pricing off the risk-free curve

Before any bond-specific adjustment, each cash flow can be discounted purely with the risk-free spot curve calibrated in `CurvesClass`:

$$MV_{\text{rf}} = \sum_{i=1}^k \frac{cf_i}{(1 + y(t_i))^{t_i}} + \frac{cf_M}{(1 + y(t_M))^{t_M}}$$

This is *not* the bond’s actual market price — it ignores credit/liquidity risk entirely. The gap between it and the real `Market_Price` from the input file is exactly what the z-spread in Step 5 is calibrated to close.

1.2.6 Step 5 — Calibrating the z-spread to match the market price

OSEM finds a single constant z added to the risk-free yield at every maturity such that the discounted cash flows reproduce the bond’s observed market price exactly:

$$MV = \sum_{i=1}^k \frac{cf_i}{(1 + y(t_i) + z)^{t_i}} + \frac{cf_M}{(1 + y(t_M) + z)^{t_M}}$$

There is no closed-form solution for z , so OSEM finds it numerically with a **bisection search** (`bisection_spread`) over a bracket $[z_{\text{start}}, z_{\text{end}}]$, which `calibrate_bond_portfolio` fixes at $[-0.2, 0.2]$.

The bracket has to contain a solution. The price falls monotonically in z , so a root exists only if MV lies between the prices at the two ends of the bracket. `bisection_spread` checks this before iterating and raises a `ValueError` naming the bond when it fails. Without that check the search never sees a sign change, walks z_{start} all the way up to z_{end} , and returns the bracket bound as though it were a calibrated spread. A market price outside the bracket usually means either that the bracket is too narrow for a genuinely distressed bond, or that `Coupon_Rate` was entered as an annualised rate (see the note on f above).

What precision measures. Once bracketed, the search stops when the bracket is narrower than `precision`, that is $(z_{\text{end}} - z_{\text{start}})/2 < \text{precision}$. This is a tolerance on the *spread*, not on the price. `calibrate_bond_portfolio` hardcodes `precision = 10-8`, which leaves a residual of order 10^{-6} in price on the bonds in this portfolio. The two bracket endpoints are the exception: each is returned straight away if it already prices within `precision` of MV .

1.2.7 Step 6 — Calibrating the whole portfolio

`calibrate_bond_portfolio` simply repeats Step 5’s bisection for every bond in the portfolio, storing one z per asset in `zspread_df`.

1.2.8 Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. The portfolio’s z -spreads are calibrated **once**, at the modelling date (`src/osem/main.py:253`).
2. In every subsequent period of the simulation, `price_bond_portfolio` is called again with that *same, fixed* z -spread but the *period’s* point on the projected yield curve (`src/osem/main.py:324`) — so the bond’s price moves only because the risk-free curve and remaining cash flows change, not because its credit spread is re-estimated.

This notebook only demonstrates the fixed-income leg in isolation; the full run also handles equities, cash, liabilities/unit-linked policies, and portfolio rebalancing each period.

1.2.9 Generating new bonds (not yet implemented)

OSEM’s documentation also describes a process for generating *new* corporate bonds at each future period (to replace maturing debt), calibrating their coupon so they price at par:

$$1 = \sum_{i=1}^k \frac{dy}{(1 + y(t_i) + c + s + ss)^{t_i}} + \frac{1}{(1 + y(t_M) + c + s + ss)^{t_M}}$$

This is not yet implemented in `BondClasses.py` / this notebook — see `Documentation/OSEM_Documentation_draft.ipynb` (cells 84–99) for the full methodology.

1.3 2. Preparation before running the notebook

1.3.1 Importing necessary external packages

```
[1]: import os
import sys
import numpy as np
import pandas as pd
import datetime as dt
from pathlib import Path
```

1.3.2 Orient base folder

```
[2]: base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
↳ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
base_folder = os.getcwd() # Get current working directory
```

1.3.3 Import functions from the main code

```
[3]: from osem.CurvesClass import Curves
from osem.ImportData import import_SWEiopa, get_corporate_bonds,
↳ get_configuration, get_settings
from osem.BondClasses import *
from osem.ConfigurationClass import Configuration
from osem.MainLoop import create_cashflow_dataframe
```

1.3.4 Configuration and parameters

```
[4]: conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

These lines of code just extract the absolute location of different files:

```
[5]: parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
bond_portfolio_file = conf.input_bond_portfolio
```

```
[6]: paramfile = pd.read_csv("Input/Parameters.csv").set_index("Parameter")
```

The parameter file is:

```
[7]: display(paramfile)
```

	Value
Parameter	
EIOPA_param_file	Input/Param_no_VA.csv

EIOPA_curves_file	Input/Curves_no_VA.csv
country	Slovenia
run_type	Risk Neutral
n_proj_years	50
Precision	1E-10
Tau	0.0001
compounding	-1
Modelling_Date	29/04/2023
liability_mode	cashflow
random_seed	42

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

```
[8]: settings = get_settings(parameters_file)
```

The run that the rest of the notebook prices against:

```
[9]: run_settings = pd.DataFrame(
    # Rendered as strings: in a mixed-type column pandas applies one float_
    ↪format to every
    # numeric value, which would show a precision of 1e-10 as 0.0
    {"Value": [str(settings.modelling_date),
               str(settings.end_date),
               str(settings.n_proj_years),
               settings.country,
               f"{settings.precision:g}",
               f"{settings.tau:g}"]},
    index=["Modelling date",
           "End of modelling window",
           "Projection years",
           "Country",
           "Precision (curve calibration)",
           "Tau"],
)

display(run_settings)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection years	50
Country	Slovenia
Precision (curve calibration)	1e-10
Tau	0.0001

1.4 3. Corporate bonds

The CorpBond object contains information about each fixed income position. This includes: * asset_id * nace * issuer * issue_date * maturity_date * coupon_rate * Comprehensive bond spread * notional_amount * frequency * recovery_rate * default_probability * units * market_price

A Python generator reads the bond portfolio file and encodes it into a dictionary based on the asset id. Each asset id contains a CorpBond object describing a single fixed income position.

```
[10]: bond_input_generator = get_corporate_bonds(bond_portfolio_file)
bond_input = {corp_bond.asset_id: corp_bond for corp_bond in
↳ bond_input_generator}
```

As an example, a single corporate bond in a CorpBond would look something like:

```
[11]: display(bond_input[1234])
```

```
CorpBond(asset_id=1234, nace='A1.4.5', issuer=None, issue_date=datetime.
↳ date(2021, 12, 3), maturity_date=datetime.date(2026, 12, 12), coupon_rate=0.
↳ 03, notional_amount=100.0, spread_country=0.0, spread_sector=0.0, zspread=0.
↳ 01, spread_stress=0.0, frequency=1, recovery_rate=0.4, default_probability=0.
↳ 03, units=100.0, market_price=94.0)
```

The CorpBondPortfolio is just a dictionary of such objects.

```
[12]: bond_portfolio = CorpBondPortfolio(bond_input)
```

The whole fixed income portfolio at a glance. Note that Annual coupon is derived, not an input: it is Coupon rate times Frequency, because the coupon rate is the rate paid on each coupon date rather than an annualised one.

```
[13]: portfolio_overview = pd.DataFrame(
    [{"Maturity": bond.maturity_date,
      "Notional": bond.notional_amount,
      "Coupon rate": bond.coupon_rate,
      "Frequency": int(bond.frequency),
      "Annual coupon": bond.coupon_rate * int(bond.frequency),
      "Units": bond.units,
      "Market price": bond.market_price}
    for bond in bond_portfolio.corporate_bonds.values()],
    index=list(bond_portfolio.corporate_bonds),
)

display(portfolio_overview)
```

	Maturity	Notional	Coupon rate	Frequency	Annual coupon	Units	\
1234	2026-12-12	100.0	0.030000	1	0.03	100.0	

2889	2028-12-12	100.0	0.025000	2	0.05	120.0
31	2025-12-03	100.0	0.003333	12	0.04	100.0
1	2030-06-30	100.0	0.005000	2	0.01	120.0
2	2035-12-31	100.0	0.010000	1	0.01	100.0
3	2026-12-12	100.0	0.030000	1	0.03	120.0
4	2028-12-12	100.0	0.025000	2	0.05	100.0
5	2025-12-03	100.0	0.003333	12	0.04	120.0
6	2026-12-12	100.0	0.030000	1	0.03	100.0
7	2028-12-12	100.0	0.025000	2	0.05	120.0
8	2025-12-03	100.0	0.003333	12	0.04	100.0
9	2026-12-12	100.0	0.030000	1	0.03	120.0
10	2028-12-12	100.0	0.025000	2	0.05	100.0
11	2025-12-03	100.0	0.003333	12	0.04	120.0
12	2026-12-12	100.0	0.030000	1	0.03	100.0
13	2028-12-12	100.0	0.025000	2	0.05	120.0
14	2025-12-03	100.0	0.003333	12	0.04	100.0
15	2026-12-12	100.0	0.030000	1	0.03	120.0
16	2028-12-12	100.0	0.025000	2	0.05	100.0
17	2025-12-03	100.0	0.003333	12	0.04	120.0
18	2026-12-12	100.0	0.030000	1	0.03	100.0
19	2028-12-12	100.0	0.025000	2	0.05	120.0
20	2025-12-03	100.0	0.003333	12	0.04	100.0
21	2026-12-12	100.0	0.030000	1	0.03	120.0
22	2028-12-12	100.0	0.025000	2	0.05	100.0
23	2025-12-03	100.0	0.003333	12	0.04	120.0
24	2026-12-12	100.0	0.030000	1	0.03	100.0
25	2028-12-12	100.0	0.025000	2	0.05	120.0
26	2025-12-03	100.0	0.003333	12	0.04	100.0
27	2026-12-12	100.0	0.030000	1	0.03	120.0
28	2028-12-12	100.0	0.025000	2	0.05	100.0
29	2025-12-03	100.0	0.003333	12	0.04	120.0
30	2030-06-30	100.0	0.005000	2	0.01	10.0

	Market price
1234	94.0
2889	92.0
31	96.0
1	90.0
2	85.0
3	94.0
4	92.0
5	96.0
6	94.0
7	92.0
8	96.0
9	94.0
10	92.0
11	96.0

12	94.0
13	92.0
14	96.0
15	94.0
16	92.0
17	96.0
18	94.0
19	92.0
20	96.0
21	94.0
22	92.0
23	96.0
24	94.0
25	92.0
26	96.0
27	94.0
28	92.0
29	96.0
30	90.0

1.5 4. Projection of cash flows

1.5.1 Importing the information about the economic environment

`import_SWEiopa()` reads the necessary data about the current yield curve. The risk free term structure it produces is what every bond cash flow is discounted against, before the bond specific z-spread is added on top. Inside OSEM, the parameters related to the yield curve are saved in the `Curves` object.

```
[14]: [maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.
      ↪EIOPA_param_file,
      ↪settings.EIOPA_curves_file, settings.country)
      # Curves object with information about term structure
      curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,
      ↪settings.modelling_date,
      ↪settings.country)
```

```
[15]: ufr = extra_param["UFR"]/100 # ultimate forward rate
      precision = float(settings.precision) # Numeric precision of the optimisation
      # Targeted distance between the extrapolated curve and the ufr at the
      ↪convergence point
      tau = float(settings.tau) # 1 basis point
```

```
[16]:
```

```

curves.set_observed_term_structure(maturity_vec=curve_country.index.
    ↪to_numpy(dtype=float), yield_vec=curve_country.values)
curves.calc_fwd_rates()
# The +1 matches main.py: the last projection period still has to be priced, so
    ↪the
# projected curve has to reach one year past the end of the modelling window.
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)

```

1.5.2 Sector spread input

The sector spread file lists the NACE classification and the spread components that go with it. In this proof of concept the spread is taken directly from the bond input file, so `spreadfile` is loaded here for reference only.

```

[17]: spreadfile = pd.read_csv("Input/Sector_Spread.csv")
      spreadfile.index = spreadfile["NACE"]
      del spreadfile["NACE"]

```

1.5.3 Cash flow projection of a bond portfolio

The basis of OSEM is cash flow simulation. The cash flows for the coupon payment and the return of the notional are simulated separately.

A list of dictionaries containing all the dates and amounts of coupon payments are produced by calling the `create_coupon_flows` function:

```

[18]: coupon_flows = bond_portfolio.create_coupon_flows(settings.modelling_date,
    ↪settings.end_date)

```

The list of dictionaries containing the return of the notional amount is produced by calling the function `create_maturity_flows`:

```

[19]: notional_flows = bond_portfolio.create_maturity_flows(terminal_date=settings.
    ↪end_date)

```

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all equities as rows. The non-zero entries then represent the value of the cash flow at that date. The first step is to calculate the unique dates for the entire portfolio of bonds. This is done by calling the `unique_dates_profiles()` function over the dates related to coupons or notional amount payments.

Both can then conveniently be represented as DataFrames.

Note that a vector of bond specific spreads is also provided as output.

```

[20]: unique_list = bond_portfolio.unique_dates_profile(coupon_flows)

```

```

[21]: unique_terminal_list = bond_portfolio.unique_dates_profile(notional_flows)

```

Using the sorted list of unique dates as column headers, the dataframes containing the information related to the cash flows can be produced.

The first dataframe contains the market price of each position. Additionally, the dataframe of zspreads is returned that helps to price the bonds using a discounted cash flow method. The last output is a dataframe containing the amount (units) of each bond in the portfolio is created.

```
[22]: [market_price_df, zspread_df, units_df] = bond_portfolio.  
      ↪init_bond_portfolio_to_dataframe(settings.modelling_date)
```

A dataframe of cash flows and notional amount payments is created:

```
[23]: # Dataframe with bond coupon cash flows  
cash_flows = create_cashflow_dataframe(coupon_flows, unique_list)  
# Dataframe with bond notional cash flows  
notional_cash_flows = create_cashflow_dataframe(notional_flows,   
      ↪unique_terminal_list)
```

A compact view of the same two dictionaries, one row per bond. The last coupon never falls after the date the notional is repaid, which is $L = \min(E, M)$ from Step 1:

```
[24]: coupon_schedule = pd.DataFrame({  
      "Coupons": {aid: len(flows) for aid, flows in coupon_flows.items()},  
      "Coupon amount": {aid: next(iter(flows.values())) for aid, flows in   
      ↪coupon_flows.items()},  
      "First coupon": {aid: min(flows) for aid, flows in coupon_flows.items()},  
      "Last coupon": {aid: max(flows) for aid, flows in coupon_flows.items()},  
      "Notional repaid": {aid: min(flows) for aid, flows in notional_flows.  
      ↪items()},  
})  
  
display(coupon_schedule)
```

	Coupons	Coupon amount	First coupon	Last coupon	Notional repaid
1234	4	3.000000	2023-12-03	2026-12-03	2026-12-12
2889	12	2.500000	2023-06-03	2028-12-03	2028-12-12
31	32	0.333333	2023-05-03	2025-12-03	2025-12-03
1	15	0.500000	2023-06-30	2030-06-30	2030-06-30
2	13	1.000000	2023-12-31	2035-12-31	2035-12-31
3	4	3.000000	2023-12-03	2026-12-03	2026-12-12
4	12	2.500000	2023-06-03	2028-12-03	2028-12-12
5	32	0.333333	2023-05-03	2025-12-03	2025-12-03
6	4	3.000000	2023-12-03	2026-12-03	2026-12-12
7	12	2.500000	2023-06-03	2028-12-03	2028-12-12
8	32	0.333333	2023-05-03	2025-12-03	2025-12-03
9	4	3.000000	2023-12-03	2026-12-03	2026-12-12
10	12	2.500000	2023-06-03	2028-12-03	2028-12-12
11	32	0.333333	2023-05-03	2025-12-03	2025-12-03

12	4	3.000000	2023-12-03	2026-12-03	2026-12-12
13	12	2.500000	2023-06-03	2028-12-03	2028-12-12
14	32	0.333333	2023-05-03	2025-12-03	2025-12-03
15	4	3.000000	2023-12-03	2026-12-03	2026-12-12
16	12	2.500000	2023-06-03	2028-12-03	2028-12-12
17	32	0.333333	2023-05-03	2025-12-03	2025-12-03
18	4	3.000000	2023-12-03	2026-12-03	2026-12-12
19	12	2.500000	2023-06-03	2028-12-03	2028-12-12
20	32	0.333333	2023-05-03	2025-12-03	2025-12-03
21	4	3.000000	2023-12-03	2026-12-03	2026-12-12
22	12	2.500000	2023-06-03	2028-12-03	2028-12-12
23	32	0.333333	2023-05-03	2025-12-03	2025-12-03
24	4	3.000000	2023-12-03	2026-12-03	2026-12-12
25	12	2.500000	2023-06-03	2028-12-03	2028-12-12
26	32	0.333333	2023-05-03	2025-12-03	2025-12-03
27	4	3.000000	2023-12-03	2026-12-03	2026-12-12
28	12	2.500000	2023-06-03	2028-12-03	2028-12-12
29	32	0.333333	2023-05-03	2025-12-03	2025-12-03
30	15	0.500000	2023-06-30	2030-06-30	2030-06-30

Cash flow dataframe with coupon amounts and dates:

[25]: `display(cash_flows.head())`

	2023-05-03	2023-06-03	2023-06-30	2023-07-03	2023-08-03	2023-09-03	\
1234	0.000000	0.000000	0.0	0.000000	0.000000	0.000000	
2889	0.000000	2.500000	0.0	0.000000	0.000000	0.000000	
31	0.333333	0.333333	0.0	0.333333	0.333333	0.333333	
1	0.000000	0.000000	0.5	0.000000	0.000000	0.000000	
2	0.000000	0.000000	0.0	0.000000	0.000000	0.000000	
	2023-10-03	2023-11-03	2023-12-03	2023-12-30	...	2029-06-30	\
1234	0.000000	0.000000	3.000000	0.0	...	0.0	
2889	0.000000	0.000000	2.500000	0.0	...	0.0	
31	0.333333	0.333333	0.333333	0.0	...	0.0	
1	0.000000	0.000000	0.000000	0.5	...	0.5	
2	0.000000	0.000000	0.000000	0.0	...	0.0	
	2029-12-30	2029-12-31	2030-06-30	2030-12-31	2031-12-31	2032-12-31	\
1234	0.0	0.0	0.0	0.0	0.0	0.0	
2889	0.0	0.0	0.0	0.0	0.0	0.0	
31	0.0	0.0	0.0	0.0	0.0	0.0	
1	0.5	0.0	0.5	0.0	0.0	0.0	
2	0.0	1.0	0.0	1.0	1.0	1.0	
	2033-12-31	2034-12-31	2035-12-31				

1234	0.0	0.0	0.0
2889	0.0	0.0	0.0
31	0.0	0.0	0.0
1	0.0	0.0	0.0
2	1.0	1.0	1.0

[5 rows x 66 columns]

Cash flow dataframe with notional amount payments and dates:

[26]: `display(notional_cash_flows.head())`

	2025-12-03	2026-12-12	2028-12-12	2030-06-30	2035-12-31
1234	0.0	100.0	0.0	0.0	0.0
2889	0.0	0.0	100.0	0.0	0.0
31	100.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	100.0	0.0
2	0.0	0.0	0.0	0.0	100.0

The extra spread due to the extra riskiness of the bond compared to a risk free instrument:

[27]: `display(zspread_df)`

	2023-04-29
1234	0.010
2889	0.010
31	0.010
1	0.005
2	0.005
3	0.010
4	0.010
5	0.010
6	0.010
7	0.010
8	0.010
9	0.010
10	0.010
11	0.010
12	0.010
13	0.010
14	0.010
15	0.010
16	0.010
17	0.010

18	0.010
19	0.010
20	0.010
21	0.010
22	0.010
23	0.010
24	0.010
25	0.010
26	0.010
27	0.010
28	0.010
29	0.010
30	0.005

1.6 5. Pricing and calibration

1.6.1 Calculation of present value of each instrument

The cashflows can be used to price the current market value of the bond, implied by the assumed economic parameters.

At this point `zspread_df` still holds the `Z_Spread` column read from the portfolio file, so the prices below are discounted at the risk free curve **plus that input spread**, not at the risk free curve alone. They therefore do not reproduce the observed `Market_Price`; closing that gap is what the calibration in the next section does.

Note that `main.py` has no counterpart to this step. There the bond price at the modelling date is the `Market_Price` from the input file, and the first discounted cash flow price is produced inside the main loop.

For simplicity, this example does the pricing at the modelling date by setting the projection year equal to 0.

```
[28]: proj_period = 0
```

The present value of the bond implied by the current yield structure is:

```
[29]: market_price_df = bond_portfolio.price_bond_portfolio(cash_flows,
↳notional_cash_flows, settings, proj_period, curves, zspread_df,
↳market_price_df, settings.modelling_date)
```

```
[30]: display(market_price_df)
```

	2023-04-29
1234	97.617543
2889	107.484140
31	99.940830
1	85.545885

2	76.049447
3	97.617543
4	107.484140
5	99.940830
6	97.617543
7	107.484140
8	99.940830
9	97.617543
10	107.484140
11	99.940830
12	97.617543
13	107.484140
14	99.940830
15	97.617543
16	107.484140
17	99.940830
18	97.617543
19	107.484140
20	99.940830
21	97.617543
22	107.484140
23	99.940830
24	97.617543
25	107.484140
26	99.940830
27	97.617543
28	107.484140
29	99.940830
30	85.545885

1.6.2 Step 4 in numbers: the gap the z-spread has to close

Pricing the same cash flows at $z = 0$ gives MV_{rf} , the value of the bond if it carried no credit or liquidity risk at all. Comparing the three columns shows what the calibration in the next section is for: the risk free price is what the cash flows are worth on the EIOPA curve alone, the market price is what the bond actually trades at, and the z-spread is the constant addition to the curve that closes the difference.

```
[31]: risk_free_price = {}
      for asset_id, bond in bond_portfolio.corporate_bonds.items():
          risk_free_price[asset_id] = bond.price_bond(cash_flows.loc[asset_id],
                                                    notional_cash_flows.
                                                    ↪loc[asset_id],
                                                    settings.modelling_date,
                                                    proj_period,
```

```

curves,
0.0)

price_comparison = pd.DataFrame({
    "Risk free price": pd.Series(risk_free_price),
    "Price at input z-spread": market_price_df.iloc[:, 0],
    "Market price": pd.Series({aid: b.market_price
                               for aid, b in bond_portfolio.corporate_bonds.
                               items()}),
})
price_comparison["Gap to close"] = (price_comparison["Risk free price"]
                                   - price_comparison["Market price"])

display(price_comparison)

```

	Risk free price	Price at input z-spread	Market price	Gap to close
1234	100.920893	97.617543	94.0	6.920893
2889	112.713514	107.484140	92.0	20.713514
31	102.345846	99.940830	96.0	6.345846
1	88.452592	85.545885	90.0	-1.547408
2	80.527229	76.049447	85.0	-4.472771
3	100.920893	97.617543	94.0	6.920893
4	112.713514	107.484140	92.0	20.713514
5	102.345846	99.940830	96.0	6.345846
6	100.920893	97.617543	94.0	6.920893
7	112.713514	107.484140	92.0	20.713514
8	102.345846	99.940830	96.0	6.345846
9	100.920893	97.617543	94.0	6.920893
10	112.713514	107.484140	92.0	20.713514
11	102.345846	99.940830	96.0	6.345846
12	100.920893	97.617543	94.0	6.920893
13	112.713514	107.484140	92.0	20.713514
14	102.345846	99.940830	96.0	6.345846
15	100.920893	97.617543	94.0	6.920893
16	112.713514	107.484140	92.0	20.713514
17	102.345846	99.940830	96.0	6.345846
18	100.920893	97.617543	94.0	6.920893
19	112.713514	107.484140	92.0	20.713514
20	102.345846	99.940830	96.0	6.345846
21	100.920893	97.617543	94.0	6.920893
22	112.713514	107.484140	92.0	20.713514
23	102.345846	99.940830	96.0	6.345846
24	100.920893	97.617543	94.0	6.920893
25	112.713514	107.484140	92.0	20.713514
26	102.345846	99.940830	96.0	6.345846
27	100.920893	97.617543	94.0	6.920893
28	112.713514	107.484140	92.0	20.713514
29	102.345846	99.940830	96.0	6.345846

30	88.452592	85.545885	90.0	-1.547408
----	-----------	-----------	------	-----------

1.6.3 Calibrate the spread to match market price

To calibrate the spread implied by the market, OSEM uses a bisection method to obtain the spread such that when added on top of the risk free term structure, the discounted cashflows equal to the current market price.

```
[32]: calibrated_spread = bond_portfolio.corporate_bonds[1234].  
      ↪bisection_spread(x_start=-0.2  
                      , x_end=0.2  
                      , modelling_date=settings.modelling_date  
                      , end_date=settings.end_date  
                      , proj_period=proj_period  
                      , curves=curves  
                      , precision= 0.00000001  
                      , max_iter=100000)
```

The market value calculated using the discounted cash flow method using the calibrated zspread is:

```
[33]: calibrated_bond = bond_portfolio.corporate_bonds[1234].price_bond(cash_flows.  
      ↪loc[1234],notional_cash_flows.loc[1234],settings.modelling_date,↪  
      ↪proj_period,curves,calibrated_spread)
```

```
[34]: print(calibrated_bond)
```

```
93.99999907078171
```

The function to calibrate the entire portfolio:

```
[35]: zspread_df=bond_portfolio.calibrate_bond_portfolio(zspread_df, settings,↪  
      ↪proj_period, curves)
```

```
[36]: display(zspread_df)
```

```
      2023-04-29  
1234    0.021478  
2889    0.043773  
31      0.027159  
1       -0.002584  
2       -0.004690  
3       0.021478  
4       0.043773
```

5	0.027159
6	0.021478
7	0.043773
8	0.027159
9	0.021478
10	0.043773
11	0.027159
12	0.021478
13	0.043773
14	0.027159
15	0.021478
16	0.043773
17	0.027159
18	0.021478
19	0.043773
20	0.027159
21	0.021478
22	0.043773
23	0.027159
24	0.021478
25	0.043773
26	0.027159
27	0.021478
28	0.043773
29	0.027159
30	-0.002584

Repricing the portfolio with the calibrated spreads reproduces the observed market prices. The residual is the price cost of stopping the bisection once the spread bracket is narrower than **precision**, as described in Step 5:

```
[37]: calibrated_price_df = bond_portfolio.price_bond_portfolio(cash_flows,
                                                             notional_cash_flows,
                                                             settings,
                                                             proj_period,
                                                             curves,
                                                             zspread_df,
                                                             market_price_df.copy(),
                                                             settings.
                                                             modelling_date)

calibration_check = pd.DataFrame({
    "Calibrated z-spread": zspread_df.iloc[:, 0],
    "Repriced value": calibrated_price_df.iloc[:, 0],
    "Market price": pd.Series({aid: bond.market_price
```

```

                                for aid, bond in bond_portfolio.corporate_bonds.
                                ↪items()}),
    })
    calibration_check["Residual"] = (calibration_check["Repriced value"]
                                     - calibration_check["Market price"])

    display(calibration_check)

```

	Calibrated z-spread	Repriced value	Market price	Residual
1234	0.021478	93.999999	94.0	-9.292183e-07
2889	0.043773	92.000000	92.0	4.150266e-07
31	0.027159	96.000000	96.0	-7.895707e-08
1	-0.002584	90.000002	90.0	1.608488e-06
2	-0.004690	84.999997	85.0	-2.740654e-06
3	0.021478	93.999999	94.0	-9.292183e-07
4	0.043773	92.000000	92.0	4.150266e-07
5	0.027159	96.000000	96.0	-7.895707e-08
6	0.021478	93.999999	94.0	-9.292183e-07
7	0.043773	92.000000	92.0	4.150266e-07
8	0.027159	96.000000	96.0	-7.895707e-08
9	0.021478	93.999999	94.0	-9.292183e-07
10	0.043773	92.000000	92.0	4.150266e-07
11	0.027159	96.000000	96.0	-7.895707e-08
12	0.021478	93.999999	94.0	-9.292183e-07
13	0.043773	92.000000	92.0	4.150266e-07
14	0.027159	96.000000	96.0	-7.895707e-08
15	0.021478	93.999999	94.0	-9.292183e-07
16	0.043773	92.000000	92.0	4.150266e-07
17	0.027159	96.000000	96.0	-7.895707e-08
18	0.021478	93.999999	94.0	-9.292183e-07
19	0.043773	92.000000	92.0	4.150266e-07
20	0.027159	96.000000	96.0	-7.895707e-08
21	0.021478	93.999999	94.0	-9.292183e-07
22	0.043773	92.000000	92.0	4.150266e-07
23	0.027159	96.000000	96.0	-7.895707e-08
24	0.021478	93.999999	94.0	-9.292183e-07
25	0.043773	92.000000	92.0	4.150266e-07
26	0.027159	96.000000	96.0	-7.895707e-08
27	0.021478	93.999999	94.0	-9.292183e-07
28	0.043773	92.000000	92.0	4.150266e-07
29	0.027159	96.000000	96.0	-7.895707e-08
30	-0.002584	90.000002	90.0	1.608488e-06

1.7 6. Appendix: input data scheme

There are multiple input files needed to calibrate the fixed income portfolio. They are located in the “Input” folder.

1.7.1 Parameters.csv

Parameters file holds information about the type of run and the modelling date.

- EIOPA_param_file ...the relative location of the EIOPA parameter file that will be used as the RFR Ex. “Input/Param_no_VA.csv”
- EIOPA_curves_file ... the relative location of the EIOPA yield curve that will be used as the RFR Ex. “Input/Curves_no_VA.csv”
- country ... the name of the country that will be used as the base for this run Ex. “Slovenia”
- n_proj_years ... length of a run in years starting from the Modelling date Ex. 50
- Precision ... precision parameter used when calibrating the risk free curve in the Curves object Ex. 1E-10. Note that the bond z-spread calibration does not use it: calibrate_bond_portfolio hardcodes a tolerance of 1e-8
- Tau ... the acceptable size of the gap between the extrapolated yield rate and the ultimate forward rate Ex. 0.0001
- compounding ... the way that the interest rates are compounded in the run Ex. -1
- Modelling_Date ... the starting date of the run specified as a date string Ex.”29/04/2023”

1.7.2 EIOPA RFR files

There are two types of files derived from the monthly EIOPA RFR submission that are used in this model. The “Curves_XX.csv” containing the yearly yield curves for all countries in scope and the “Param_XX.csv” with the parameters used to derive the curves. These files are used to derive the risk free term structure at the modelling date and to efficiently project the evolution of the term structure.

1.7.3 Portfolio description

The modelled portfolio is split by asset classes. The fixed income portfolio is located in the file “Bond_Portfolio.csv”. Each security needs the following fields:

- Asset ID ... unique id such as an ISIN, SEDOL or CUSIP code Ex. IT1234567891
- Asset_Type ... asset type string Ex. “Corporate_Bond”
- NACE ... NACE asset classification code (nomenclature statistique des activités économiques dans la Communauté européenne) Ex. A14.5
- Issue_Date ... the string date specifying the issue date of the bond Ex. 3/12/2021
- Maturity_Date ... the string date specifying the maturity date of the bond Ex. 3/12/2021
- Notional_amount ... the notional amount of the bond Ex. 100
- Coupon_Rate ... percentage of the notional amount paid in dividends every period (specified by Frequency) Ex. 0.0014
- Frequency ... number of times per a year that dividends are paid Ex. 1 (once per a year)
- Recovery_Rate ... percentage of the notional amount that can be recovered in case of a default Ex. 0.80
- Default_Probability ... percentage probability of default per year Ex. 0.012
- Units ... number of each bond held in the portfolio Ex. 230

- Market_Price ... market price of the bond at the modelling date Ex. 96

1.7.4 Sector spread

The list of NACE sector codes and the sector specific spread over the risk free rate - NACE ... NACE code of the issuer Ex. "A1.1" - NACE code text ... description of the NACE code for this issuer Ex. "Growing of non-perennial crops" - cSpread ... Country of issuance or operations specific spread Ex. 0.01 - sSpread ... Sector specific spread Ex. 0.01 - zSpread ... bond issuance sector specific spread Ex. 0.01 - ssSpread ... Extra spread assumed by the specific stress scenario Ex. 0.01

In the POC, the spread is displayed directly in the bond input file